

Postgres DBA Basics Nobody Told You

Session 2 of 3 — Beginning Postgres Workshop

PGSummit NYC 2026

Session 2 Topics

- WAL: how Postgres stays durable
- Backups: dump/restore, basebackup, point-in-time recovery
- Upgrades: minor and major versions
- Replication: streaming & logical (concepts)
- Connection pooling
- Disk, storage, and vacuum
- Extensions

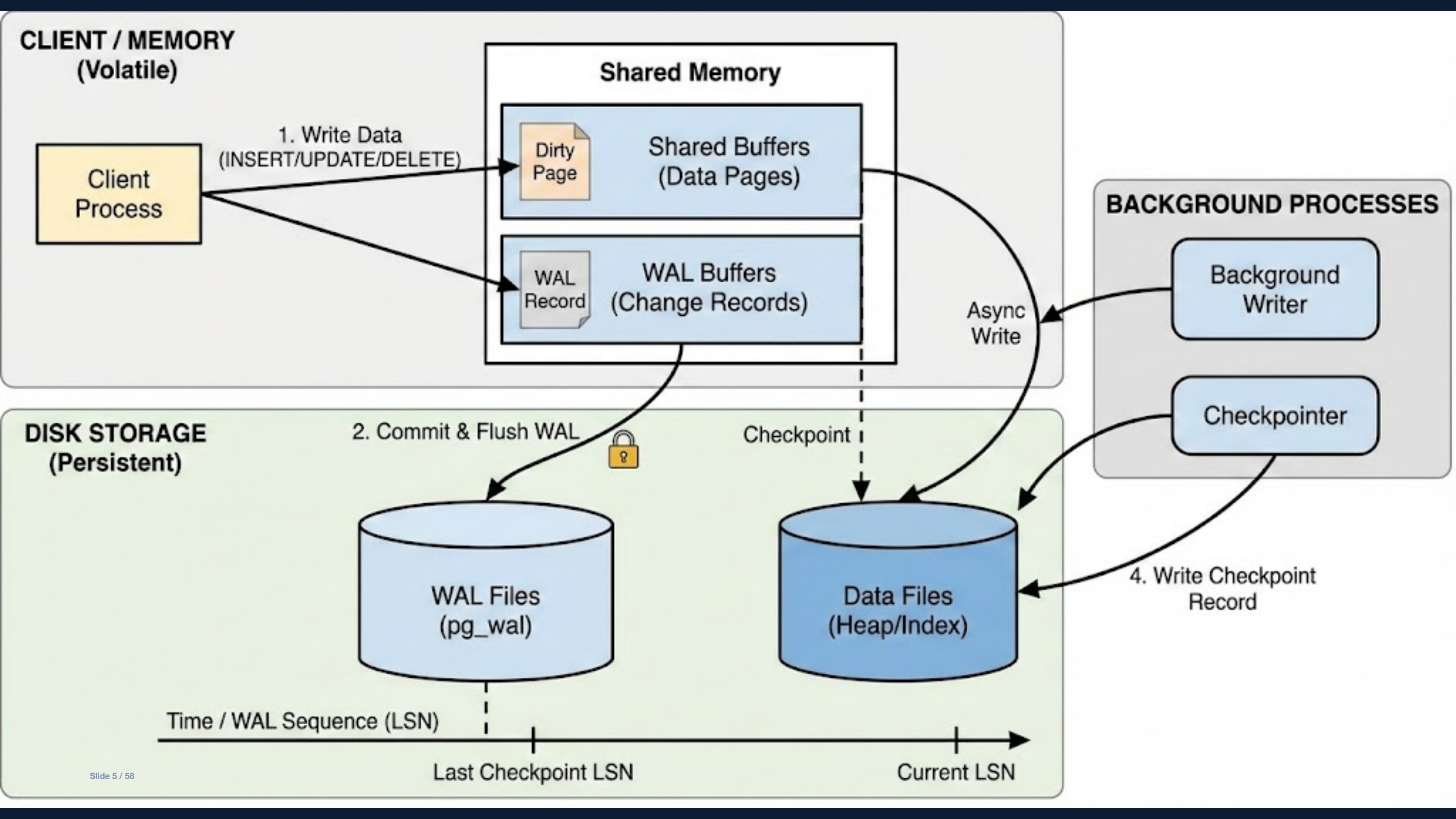
Understanding WAL

Write-Ahead Logging

What is WAL?

Write-Ahead Logging - PostgreSQL's durability mechanism

1. Before data is written to tables, changes are logged to WAL
2. WAL is sequentially written (fast!)
3. On crash, WAL is "replayed" to recover data
4. WAL is also used for replicas, HA scenarios, and robust PITR backup systems



Demo: View WAL

```
-- Current WAL position (Log Sequence Number)
SELECT pg_current_wal_lsn();
-- 0/3A8B9D0
```

```
-- WAL stats
SELECT wal_records, wal_bytes,
       pg_size_pretty(wal_bytes) as wal_size
FROM pg_stat_wal;
```

```
## List WAL files in the container
docker exec postgres-training ls -la \
    /var/lib/postgresql/18/docker/pg_wal/
```

Backups

Backup Strategy Fundamentals

- **What** to backup: data, configs, WAL
- **Where** to store: local, remote, cloud
- **How often**: RPO (Recovery Point Objective)
- **How fast** to recover: RTO (Recovery Time Objective)

Backup Choices

- Dump/restore - simple, but not automated, long restore times
- Basebackup with added backups - more robust but self managed
- Automated backup tools (see the end of this section) - robust, reliable, added complexity

pg_dump - a copy, not a real backup

Note: must have PG 18 installed for this to work locally

Dump entire database (using connection string)

```
pg_dump postgresql://postgres:training@localhost:5432/bluebox > bluebox.sql
```

Dump in custom format (compressed, parallel restore)

```
pg_dump -Fc postgresql://postgres:training@localhost:5432/bluebox > bluebox.dump
```

Dump specific tables

```
pg_dump -t 'bluebox.film' -t 'bluebox.rental' \  
    postgresql://postgres:training@localhost:5432/bluebox > tables.sql
```

pg_dump Options

Option	Description
<code>-Fc</code>	Custom format (recommended)
<code>-Fd</code>	Directory format (parallel)
<code>-j 4</code>	Parallel jobs
<code>--schema-only</code>	Structure only, no data
<code>--data-only</code>	Data only, no structure
<code>-t tablename</code>	Specific table

pg_dumpall - All Databases

Dump all databases including globals (roles, tablespaces)

```
pg_dumpall -d postgresql://postgres:training@localhost:5432/postgres > full_cluster.sql
```

Globals only (roles, tablespaces)

```
pg_dumpall --globals-only -d postgresql://postgres:training@localhost:5432/postgres > globals.sql
```

pg_restore - Restoring Backups

Restore from custom format

```
pg_restore -h localhost -U postgres -d bluebox_new bluebox.dump
```

Parallel restore

```
pg_restore -j 4 -h localhost -U postgres -d bluebox_new bluebox.dump
```

Restore specific table

```
pg_restore -t film -h localhost -U postgres -d bluebox_new bluebox.dump
```

pg_basebackup - Physical Backup

Copies the entire database cluster at the file level.

Unlike pg_dump (logical), pg_basebackup:

- Copies raw data files
- Faster for large databases
- Required for streaming replication setup
- Enables point-in-time recovery

Demo: pg_basebackup

Let's take a real backup in our Docker environment:

```
## Create backup directory inside container
docker exec postgres-training mkdir -p /backup

## Take a full backup (plain format)
## Backup goes to /backup/full INSIDE the container
docker exec postgres-training pg_basebackup \
    -U postgres \
    -D /backup/full \
    -Fp \
    -Xs \
    -P
```

Note: This backup is inside the container. In production, you'd mount an external volume or copy backups out.

pg_basebackup Options

Option	Description
-D	Destination directory
-Fp	Plain format (directory of files)
-Ft	Tar format (single archive)
-Xs	Stream WAL during backup
-Xf	Fetch WAL after backup
-P	Show progress
-c fast	Fast checkpoint (don't wait)

Inspect the Backup

List backup contents

```
docker exec postgres-training ls -la /backup/full/
```

You'll see the PostgreSQL data directory structure:

base/ – Table data files

global/ – Cluster-wide tables

pg_wal/ – WAL files included in backup

backup_manifest – Backup metadata (v13+)

Verify the Backup Manifest

```
## View the backup manifest (JSON)
docker exec postgres-training head -50 /backup/full/backup_manifest
```

```
{ "PostgreSQL-Backup-Manifest-Version": 2,
  "System-Identifier": "7470012345678901234",
  "Files": [
    { "Path": "backup_label", "Size": 225,
      "Checksum": "abc123..." },
    ...
  ]
}
```

The manifest lists every file with checksums for verification, and `pg_verifybackup` checks the backup against it:

```
docker exec postgres-training pg_verifybackup /backup/full
## backup successfully verified
```

Why WAL Matters for Backups: Consistency

- changes are made to the database during the filesystem copy during pg_basebackup
- without WAL, your copy will have files from before/after changes exist
- no guarantees of consistency; database needs to recover to a known checkpoint
- **a data directory copy without WAL generated during the dump IS NOT A BACKUP!**

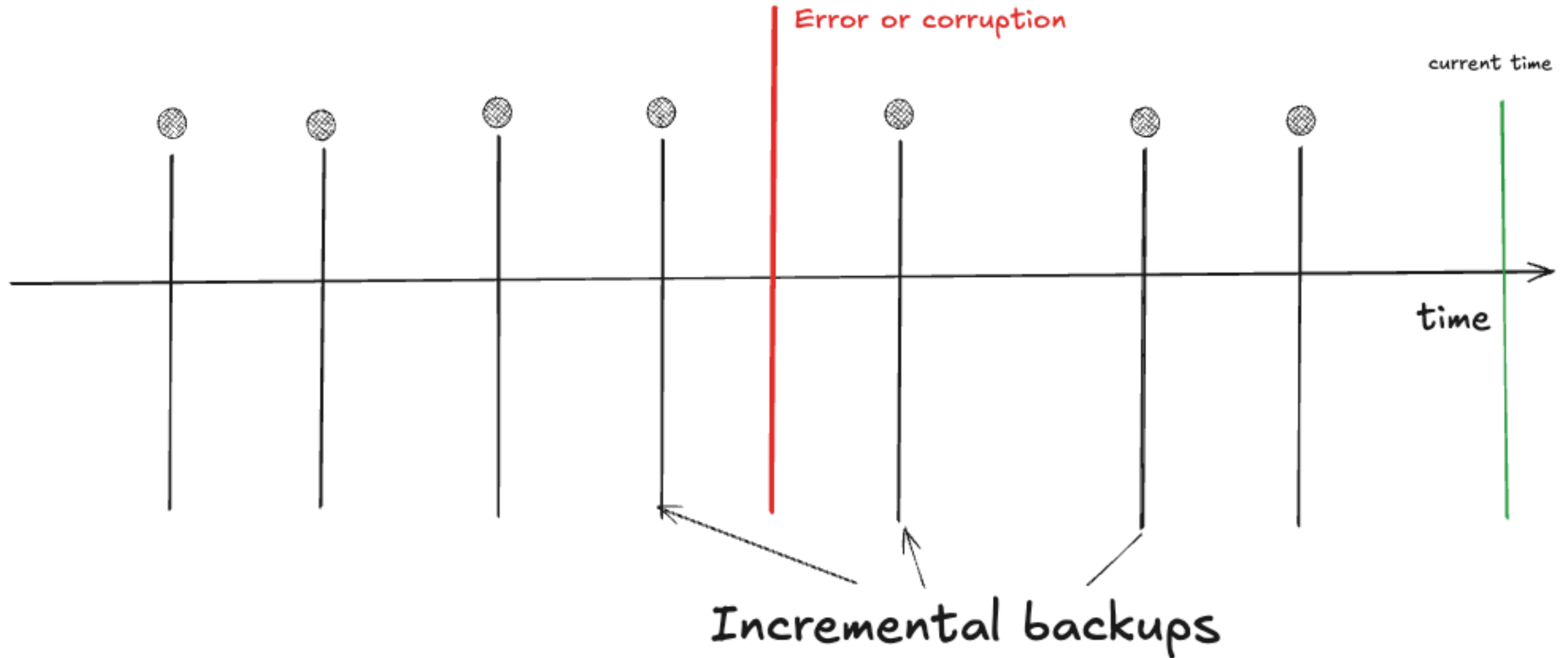
Point-in-Time Recovery (PITR)

WAL enables **Point-in-Time Recovery (PITR)**

Base Backup (Monday) + WAL files = Any point in time

- Take a base backup occasionally (daily/weekly)
- Archive WAL files continuously
- Recover to any moment: "Restore to Tuesday 2:47 PM"

Postgres Point in Time Recovery



Backup Strategy Example

Small database (< 100 GB)

- Daily pg_basebackup + continuous WAL archiving (or let a tool like pgBackRest do both)
- Optional nightly pg_dump as a logical copy for single-table restores
- Keep 7 days of backups

Large database (> 100 GB)

- Weekly full pg_basebackup
- Daily incremental backups (v17+)
- Continuous WAL archiving for PITR
- Keep 4 weeks + monthly archives

Physical Backup Summary

1. **pg_basebackup** - Takes full cluster copy
2. **backup_manifest** - Tracks files and checksums
3. **WAL archiving** - Enables point-in-time recovery
4. **Incremental (v17+)** - Only changed blocks;
pg_combinebackup merges the chain at restore time
5. **pg_verifybackup** - Validates backup integrity

Other Backup Tools

- **pgBackRest** - Parallel backup, retention policies, encryption
- **Barman** - Backup and recovery manager
- **WAL-G** - Cloud-native archival

Upgrades and Versions

Postgres Today

One major release a year, five years of support each — here's where that puts us:

PG 14	PG 15	PG 16	PG 17	PG 18	PG 19
 EOL in weeks	 Supported	 Supported	 Supported	 Current stable	 Public beta

Full dates on the next slide →

Current Support Status

Version	First Release	End of Life
19	Public beta now	~Nov 2031
18	Sept 2025	Nov 2030
17	Sept 2024	Nov 2029
16	Sept 2023	Nov 2028
15	Oct 2022	Nov 2027
14	Sept 2021	Nov 12, 2026

PG 14 goes end-of-life six weeks after this conference. PG 19 is in public beta (beta 4) now, with general availability expected this fall.

Minor Version Upgrades

Simple - just update packages and restart

```
## Ubuntu/Debian
```

```
apt update && apt upgrade postgresql-18
```

```
## Restart
```

```
systemctl restart postgresql
```

Always read the release notes!

- may have special instructions for proper upgrade



Demo: Minor Upgrade with Docker

```
## Check current version
docker exec postgres-training psql -U postgres \
  -c "SELECT version();"
```

```
## Stop containers
docker compose down
```

```
## Pull latest image (gets newest 18.x)
docker compose pull
```

```
## Start with new image – data volume persists
docker compose up -d
```

```
## Verify new version
docker exec postgres-training psql -U postgres \
  -c "SELECT version();"
```

Major Version Upgrades - Options

1. **pg_dump/pg_restore** - Logical, works across versions
2. **pg_upgrade** - In-place, faster for large databases
3. **Logical replication** - Zero/minimal downtime

pg_upgrade

Stop both clusters

```
systemctl stop postgresql
```

Run pg_upgrade

```
pg_upgrade \  
-b /usr/lib/postgresql/17/bin \  
-B /usr/lib/postgresql/18/bin \  
-d /var/lib/postgresql/17/main \  
-D /var/lib/postgresql/18/main
```

Start new cluster

```
systemctl start postgresql@18-main
```

Pre-Upgrade Checklist

- ☐ Test upgrade in non-production first
- ☐ Review release notes for breaking changes
- ☐ Check extension compatibility
- ☐ Verify disk space (2x database size)
- ☐ Plan maintenance window
- ☐ Script/document all necessary changes
- ☐ Have rollback plan ready

DR & HA Concepts

Key Terms

- **HA** - High Availability (minimize downtime)
- **DR** - Disaster Recovery (survive catastrophe)

RPO and RTO from the backup section apply here too: failover shrinks RTO, synchronous replication shrinks RPO.

Streaming Replication

Primary server streams WAL to standby

Primary — WAL Stream —> Standby (Hot Standby)

- **Synchronous:** Zero data loss, higher latency
- **Asynchronous:** Some data loss risk, lower latency
- server can be setup as Async for speed, but individual transactions can be sync for protection

Setting Up Streaming Replication

Primary `postgresql.conf` (plus a `REPLICATION` role and a `pg_hba.conf` entry for it):

```
wal_level = replica  
max_wal_senders = 3  
wal_keep_size = 1GB
```

Create the standby from a base backup of the primary:

```
pg_basebackup -h primary -U replicator -D /var/lib/postgresql/18/main -R -Xs -P
```

`-R` writes `primary_conninfo` and `standby.signal` for you. Start the standby and it streams WAL from the primary; `hot_standby = on` (the default) allows read queries on it.

Failover Options

Manual

- Promote standby manually
- Update application connections
- Simple but slow

Automatic

- Patroni
- repmgr
- pgpool-II
- Cloud provider HA

Patroni - HA Solution

```
## patroni.yml
scope: postgres-cluster
name: node1

postgresql:
  listen: "*:5432"
  data_dir: /data/postgresql

etcd:
  hosts: etcd1:2379,etcd2:2379,etcd3:2379
```

Hands-on Patroni is out of scope, but try the official demo:

github.com/patroni/patroni (see docker-compose.yml)

Logical Replication

What is Logical Replication?

Replicates data changes at the logical level (SQL)

Unlike streaming replication:

- Can replicate only certain tables instead of entire cluster
- Can replicate between different versions
- Subscriber can have writes

Logical Replication Use Cases

- Zero-downtime major upgrades
- Consolidating data from multiple sources
- Selective replication (specific rows, types of updates)
- Real-time reporting replicas
- Multi-region distribution
- OS/server architecture differences

Connection Management

The Connection Problem

Each Postgres connection = 1 process

- Memory overhead (~5-10MB per connection)
- Context switching costs
- Applications leave idle connections
- max_connections has practical limits



Demo: Finding Idle Connections

```
SELECT pid, username, state, query_start, state_change  
FROM pg_stat_activity  
WHERE state = 'idle'  
ORDER BY state_change;
```

Solution: Connection pooling

PgBouncer - Connection Pooler

PgBouncer sits between your app and PostgreSQL:

App (1000 connections) → PgBouncer → PostgreSQL (20 connections)

- Lightweight (single process, low memory)
- Reuses database connections across clients
- Our Docker Compose includes it: `docker compose --profile dba up -d` starts it on port 6432

Disk, Storage, and Vacuum

MVCC - Multi-Version Concurrency Control

PostgreSQL keeps old row versions for:

- Read consistency
- Transaction rollback
- No read locks on writes

But: Dead rows accumulate!

What is Vacuum?

VACUUM reclaims space from dead rows

-- Manual vacuum

```
VACUUM bluebox.rental;
```

-- Vacuum with analysis

```
VACUUM ANALYZE bluebox.rental;
```

-- Full vacuum (rewrites table, locks!)

```
VACUUM FULL bluebox.rental;
```

Autovacuum

Postgres automatically vacuums tables

-- Key settings

autovacuum = on

autovacuum_vacuum_threshold = 50

autovacuum_vacuum_scale_factor = 0.2

-- 20% of rows changed + 50 = trigger vacuum

Monitoring Table Bloat

```
-- Check dead tuples
SELECT
    schemaname,
    relname,
    n_dead_tup,
    n_live_tup,
    round(n_dead_tup::numeric / NULLIF(n_live_tup, 0) * 100, 2) as dead_pct
FROM pg_stat_user_tables
WHERE n_dead_tup > 0
ORDER BY n_dead_tup DESC
LIMIT 10;
```

Table and Index Bloat

```
-- pgstattuple is a contrib extension (more on extensions at the end of this session)
CREATE EXTENSION pgstattuple;

SELECT * FROM pgstattuple('bluebox.rental');
```

Tools for bloat analysis:

- check_postgres (Nagios plugin)
- pgstattuple extension
- Various community queries

Disk Space Monitoring

```
SELECT pg_size_pretty(pg_database_size('bluebox'));
```

```
-- Result: 876 MB
```

```
SELECT relname, pg_size_pretty(pg_total_relation_size(relid)) as size
FROM pg_stat_user_tables WHERE schemaname = 'bluebox'
ORDER BY pg_total_relation_size(relid) DESC LIMIT 5;
```

relname	size
rental	286 MB
payment	181 MB
inventory	162 MB
person	54 MB
film_crew	45 MB

Extensions

What are Extensions?

Extensions add functionality to Postgres:

- New data types
- New functions
- New operators
- New index types

Contrib Extensions

Bundled with Postgres - just `CREATE EXTENSION name;`

Extension	What it does
<code>pg_stat_statements</code>	Track query performance statistics
<code>pg_trgm</code>	Fuzzy text search with trigrams
<code>citext</code>	Case-insensitive text type
<code>hstore</code>	Key-value store in a column
<code>btree_gist</code>	GiST index support for common types
<code>uuid-osp</code>	Generate UUIDs
<code>tablefunc</code>	Crosstab / pivot table queries
<code>pgcrypto</code>	Encryption functions

Popular Third-Party Extensions

- **PostGIS** - Geospatial data and queries
- **pgvector** - Vector similarity search (AI/ML)
- **TimescaleDB** - Time-series data
- **pg_cron** - Job scheduling
- **HypoPG** - Hypothetical indexes

Session 2 Summary

-  WAL: durability, archiving, PITR
-  Backup strategies: pg_dump, basebackup, tools
-  Minor and major version upgrades
-  HA/DR concepts: streaming replication, failover
-  Logical replication: what it is and when to use it
-  Connection pooling with PgBouncer
-  VACUUM and storage management
-  Extensions: contrib and popular third-party

Questions?

**Next: When Postgres Misbehaves — Locks,
Monitoring, Key Config & Reading Query Plans**